AI-BASED ROAD DAMAGE AND POTHOLE DETECTION SYSTEM

A report submitted for the course of

APPLICATION DEVELOPMENT – PROJECT EXPLORE

IV B. Tech II Semester

By

K. LAKSHMI SAI CHINMAYEE – 2211IT010045
D. SAI SINDHU - 2211IT010034
J. ARUN - 2211IT010044
K. SANDEEP - 2211IT010050

Under the esteemed guidance of

Dr. L. Neelu
Associate Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

Malla Reddy University
Maisammaguda, Dulapally, Hyderabad, Telangana 500100
www.mallareddyuniversity.ac.in

2025-26

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that this bonafide record of the Application Development entitled **"AI-BASED ROAD DAMAGE AND POTHOLE DETECTION SYSTEM"** submitted **by Ms. K. Lakshmi Sai Chinmayee 2211IT010045** , **Ms. D. Sai Sindhu 2211IT010034**, **Mr. J. Arun 2211IT010044**, **Mr. K. Sandeep 2211IT010050**,of **IV** year **II** semester to the **Malla Reddy University**, Hyderabad. This bonafide record of work carried out by us under the guidance of our supervision. The contents of this report, in full or in parts, have not been submitted to any other Organization for the award of any Degree.

Internal Guide:
Dr. L. Neelu
Associate Professor

Dean of IT Department
Dr. A. V. L. N. Sujith

External Examiner

Date:

DECLARATION

We certify that

a. The work contained in this report is original and has been done by us under the guidance of our supervisor.

b. The work has not been submitted to any other Organization for any degree.

c. We have followed the guidelines provided by the Organization in preparing the report.

d. We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Organization.

e. Whenever we have used materials (data, theoretical analysis, figures, and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references.

K. Lakshmi Sai Chinmayee	**- 2211IT010045**
D. Sai Sindhu	**- 2211IT010034**
J. Arun	**- 2211IT010044**
K. Sandeep	**- 2211IT010050**

ACKNOWLEDGEMENT

We would like to take this opportunity to acknowledge everyone who has helped us in every stage of this project.

We ineffably indebted **Dr. L. Neelu**, Associate Professor, Dept. of IT for conscientious guidance and encouragement to accomplish this Application.

We ineffably indebted **Mr. S. Ram Pavan Sai**, Assistant Professor, Dept. of IT for coordinating to accomplish this Application.

We wish to convey our sincere thanks to **Dr. A. V. L. N. Sujith, Dean, Dept. of IT, Malla Reddy University** for his continuous support.

We would like to express our special gratitude and thanks to **Dr. VSK Reddy, Vice Chancellor, Malla Reddy University** for his continuous support.

We are also greatful to all administrative staff and technical staff of Information Technology Department, people who provided us the useful and necessary information needed for this project.

ABSTRACT

The rapid development of Artificial Intelligence (AI) and Computer Vision technologies has significantly improved automated monitoring systems in infrastructure management. Road damage and potholes are major contributors to traffic accidents, vehicle damage, and transportation inefficiencies. Manual road inspection methods are time-consuming, labor-intensive, and often lack real-time responsiveness. This project, titled **"AI-Based Road Damage and Pothole Detection System,"** aims to design and develop an intelligent system capable of automatically detecting potholes, classifying their severity, and notifying users in real time. The primary objective of the system is to enhance road safety, enable efficient maintenance prioritization, and provide automated monitoring without human intervention.

The proposed system is implemented using Python and integrates advanced deep learning models for detection and classification. The system uses a YOLO-based object detection model to accurately localize potholes in road images through bounding box predictions with confidence score filtering. After detection, the identified pothole region is cropped and passed to a Convolutional Neural Network (CNN) model for multi-class severity classification into Low, Medium, and High categories. The workflow involves image preprocessing (resizing and normalization), detection, severity estimation, and structured result generation.

The system supports two operational modules. The first module allows users to upload road images through a web-based interface, where the system processes the image and displays detection results along with severity classification. The second module enables automated real-time detection while a user travels on the road. In this mode, the system continuously analyzes captured frames, detects potholes, and sends notifications to the user upon identification of road damage. The backend inference orchestration is handled using FastAPI, while MongoDB is used for persistent storage of detection metadata and severity outputs. Aggregated analytics are generated to support severity-based maintenance prioritization.

Unlike many existing solutions that focus only on detection, this project integrates pothole detection, severity classification, real-time notifications, and database-driven analytics within a fully local deployment architecture. The system ensures practical implementation, stable performance, and reliable end-to-end processing without inference failure.

.

CONTENTS

Certificate		II
Declaration		III
Acknowledgement		IV
Abstract		V
Contents		VI
List of Figures		VIII
Chapter 1	**Introduction**	9
	1.1 Introduction	9
	1.2 Scope of the Project	10
	1.3 Aim of the Project	11
	1.4 Literature Survey	12-13
Chapter 2	**System Architecture**	14
	2.1 System Designing	13
	2.1.1 Software Requirements	13
	2.1.2 Hardware Requirements	14
	2.1.3 Functional Requirements	15
	2.1.4 Non-functional Requirements	16
Chapter 3	**Methodology**	17
	3.1 Data Preprocessing	17
	3.2 Feature Identification and Categorization	18
	3.2.2 Connection Information	19
	3.2.3 Connection Content	19
	3.2.3 Traffic information	20
	3.3 Output Label	20
	3.4 Model Development	20
	3.5 Domain Analysis	21
Chapter 4	**Existing System**	22
	4.1 Existing Models	22
	4.1.1 Evaluation Metrics	23
	4.2 Existing vs Proposed Systems	23
	4.3 Existing System	24
	4.4 Proposed System	24
Chapter 5	**UML Designing**	25
	5.1 Unified Modeling Language	25
	5.2 Building Blocks of UML	25
	5.2.1 Things in the UML	25
	5.3 UML Diagrams	26-30
Chapter 6	**Results And Discussions**	31
	6.1 Input Design	31
	6.1.2 Input Data features	31
	6.1.3 Input Validations and	32

	Constraints	
	6.2 Output Design	33
	6.2.1 Key output features	33
	6.2.2 User Interface & Dashboard Features	33
	6.3 Results and description	34-36
Chapter 7	**Conclusion and Future of the Scope**	37
References		38
Appendix		39-44

LIST OF FIGURES

Figure No	Description	Page No
Figure 1.2	Class Diagram	26
Figure 5.3.2	Activity Diagram	27
Figure 5.3.3	Use Case Diagram	28
Figure 5.3.4	Sequence Diagram	29
Figure 1.3	Results and Description	31
Figure 6.8	Citizen login	34
Figure 6.2	Pothole Reporting	35
Figure 6.3	Image Uploading	35
Figure 6.4	Pothole Detection	36
Figure 6.5	Authority Dashboard	36

CHAPTER 1
Introduction

1.1 INTRODUCTION

Artificial Intelligence (AI) and Computer Vision have become powerful technologies that enable machines to analyze images and make intelligent decisions. These technologies are widely used in smart transportation and infrastructure monitoring systems. One major problem in road infrastructure is the presence of potholes and road damage, which can lead to accidents, vehicle damage, and traffic congestion.

Traditional road inspection methods rely on manual surveys, which are time-consuming, costly, and not suitable for real-time monitoring. With increasing traffic and expanding cities, there is a strong need for an automated system that can detect road damage quickly and accurately. An AI-based solution can help authorities monitor roads efficiently and improve maintenance planning.

The project titled **"AI-Based Road Damage and Pothole Detection System"** focuses on developing an intelligent system that automatically detects potholes from road images and classifies their severity levels. The main objective of the system is to improve road safety by identifying potholes and providing timely alerts to users.

The system consists of two main modules. The first module allows users to upload road images through a web interface, where the system detects potholes using a YOLO-based object detection model and classifies their severity (Low, Medium, High) using a CNN model. The second module performs automatic detection while a user is traveling on the road, continuously analyzing road images and sending notifications when potholes are detected.

The system is developed using Python along with deep learning frameworks and image processing libraries. FastAPI is used for backend processing, MongoDB is used for storing detection results, and a web-based interface is designed for user interaction. The architecture is fully locally deployable and does not depend on cloud infrastructure.

Although the current system uses 2D image-based analysis, it provides a strong foundation for future enhancements such as GPS integration and smart city monitoring systems. Overall, this project demonstrates how AI can be practically applied to improve road safety and infrastructure management.

The system is developed using Python along with deep learning frameworks and image processing libraries. FastAPI is used for backend processing, MongoDB is used for storing detection results, and a web-based interface is designed for user interaction. The architecture is fully locally deployable and does not depend on cloud

Although the current implementation primarily relies on 2D image-based analysis, it establishes a strong foundation for future enhancements such as GPS-based location tracking, smart city integration, real-time municipal monitoring, and advanced deep learning model improvements. Overall, this project demonstrates the practical application of artificial intelligence in intelligent road infrastructure monitoring and contributes to the development of safer and smarter transportation systems.

1.2 SCOPE OF THE PROJECT

1.FunctionalScope
The project focuses on developing an AI-based system capable of automatically detecting potholes from road images and classifying their severity levels. The system supports two main functionalities: user image upload for pothole detection and real-time road monitoring that automatically detects potholes during travel and sends notifications to the user.

2. Technical Scope:
The system is implemented using Python and integrates deep learning models such as YOLO for pothole detection and CNN for severity classification. It demonstrates the practical application of Artificial Intelligence, Computer Vision, image preprocessing, backend integration using FastAPI, and database management using MongoDB.

3. Data Management Scope:
The project includes structured storage of detection results, confidence scores, and severity classifications in a MongoDB database. This enables record maintenance, result tracking, and generation of severity-based analytics for maintenance prioritization.

4. Performance and Usability Scope:
The system is designed for efficient local deployment without cloud dependency. It ensures stable real-time inference, reliable detection results, and a user-friendly web interface for uploading images and visualizing outputs, making it suitable for academic and practical applications.

Analytics and Monitoring Scope:
The project supports aggregated analytics generation based on severity levels, which helps in identifying high-risk areas and prioritizing road maintenance activities. This makes the system useful for smart infrastructure monitoring.

Future Expansion Scope:
The system provides a foundation for future enhancements such as GPS-based location tagging, smart city integration, municipal monitoring dashboards, edge device deployment, and advanced deep learning model improvements for higher accuracy and robustness.

1.3 AIM OF THE PROJECT

Aim of the Project

The primary aim of the project **"AI-Based Road Damage and Pothole Detection System"** is to design and develop an intelligent deep learning-based system that can automatically detect potholes from road images and classify their severity levels. The project seeks to enhance road safety and infrastructure monitoring by reducing dependency on manual inspection methods and introducing an automated, accurate, and real-time detection mechanism.

This project aims to create a practical and efficient system capable of identifying potholes using a YOLO-based object detection model and further classifying them into Low, Medium, and High severity categories using a Convolutional Neural Network (CNN). The system supports two operational modes: one where users upload road images for analysis, and another where potholes are automatically detected during travel, triggering notifications to alert the user. By automating detection and severity estimation, the system intends to improve road safety, assist in maintenance prioritization, and minimize accidents caused by road damage.

Another important aim of the project is to demonstrate the practical application of Artificial Intelligence, Computer Vision, and deep learning technologies using Python. The system integrates image preprocessing, real-time inference, backend processing with FastAPI, and structured data storage using MongoDB. It emphasizes stable performance, local deployment without cloud dependency, and a modular architecture that ensures scalability and ease of maintenance.

The project also aims to provide a foundation for future advancements in smart transportation systems. Although the current implementation focuses on 2D image-based detection and classification, it is structured in a way that allows future enhancements such as GPS-based location tracking, smart city integration, municipal monitoring dashboards, and deployment on edge devices. This ensures that the system can evolve into a more intelligent and large-scale infrastructure monitoring solution.

.

1.4 LITERATURE SURVEY

Title: Real-Time Detection of Road Damages Using YOLOv8
Journal: Journal of Emerging Technologies and Innovative Research (JETIR)
Volume & Issue: Vol. 11, Issue 12
Publication Date: December 2024
DOI: 10.17148/IARJSET.2022.9216
Institution: Department of Computer Engineering, East Godavari District, Andhra Pradesh, India
Authors: Dasu Dasari, D.N.V.S. Vijaya Lakshmi, Tatapudi Ashok

The paper titled "Real-Time Detection of Road Damages Using YOLOv8: An Innovative Deep Learning Approach" presents the development of an automated road damage detection system using the YOLOv8 deep learning model. The system is designed to detect potholes and surface cracks in real time, reducing the need for manual road inspections. The model leverages high-speed object detection capabilities combined with improved accuracy. The research emphasizes the use of data augmentation techniques to enhance performance under different lighting and weather conditions. The architecture follows a pipeline including image acquisition, preprocessing, model inference, damage classification, and result visualization. The system significantly improves detection speed and accuracy, making it suitable for smart-city applications. The study suggests further enhancements through integration with GIS mapping and real-time alert systems for government authorities.

Title: YOLOv8-Based Visual Detection of Road Hazards
Journal: 2023 IEEE Pune Section International Conference (PuneCon) / arXiv
Volume & Issue: IEEE Xplore / arXiv:2311.00073
Publication Date: November 2023
DOI: 10.48550/arXiv.2311.00073
Institution: Department of Computer Engineering, Vishwakarma Institute of Information Technology, Pune, India
Authors: Om M. Khare, Shubham Gandhi, Sunil Mane

The paper titled "YOLOv8-Based Visual Detection of Road Hazards: Potholes, Sewer Covers, and Manholes" evaluates the effectiveness of YOLOv8 in detecting multiple road hazards. The system is trained on a diverse dataset to identify potholes, sewer covers, and manholes in complex real-world environments. The study compares YOLOv8 with earlier versions such as YOLOv5 and YOLOv7 and demonstrates improved precision, recall, and detection speed. The architecture follows a structured workflow including dataset preparation, model training, validation, testing, and real-time deployment. The model performs multi-class detection efficiently and handles scale variations and occlusions effectively. The research concludes that YOLOv8 provides superior performance for intelligent transportation and smart infrastructure monitoring systems.

Title: YOLOv8-Based Visual Detection of Road Hazards: Potholes, Sewer Covers, and Manholes
Journal: IEEE Pune Section International Conference (PuneCon) / arXiv
Volume & Issue: IEEE Xplore / arXiv:2311.00073
Publication Date: November 2023
DOI: 10.48550/arXiv.2311.00073
Institution: Vishwakarma Institute of Information Technology, Pune, India
Authors: Om M. Khare, Shubham Gandhi, Sunil Mane

The paper titled *"YOLOv8-Based Visual Detection of Road Hazards: Potholes, Sewer Covers, and Manholes"* presents a deep learning-based framework for automatic road hazard detection using the YOLOv8 object detection model. The primary objective of the study is to enhance road safety and infrastructure monitoring by identifying common road defects in real-time using computer vision techniques.

The authors developed a multi-class object detection system capable of identifying potholes, sewer covers, and manholes from road images and video streams. The dataset used for training includes images captured under diverse real-world conditions such as varying lighting, weather changes, shadows, traffic disturbances, and different road textures. Data preprocessing techniques such as resizing, normalization, and annotation were performed to prepare the dataset for training.

CHAPTER 2
System Architecture

2.1 SYSTEM DESIGNING

2.1.1 SOFTWARE REQUIREMENTS

Python 3.x – Core programming language used to implement the pothole detection and severity classification system.

YOLO (You Only Look Once) Framework – Used for real-time pothole detection and bounding box localization.

Convolutional Neural Network (CNN) – Used for multi-class severity classification (Low, Medium, High).

OpenCV Library – Used for image preprocessing, resizing, normalization, and frame processing in real-time detection.

FastAPI – Used to handle backend inference orchestration and API communication between frontend and models.

MongoDB – Used for persistent storage of detection metadata, severity results, and analytics data.

HTML, CSS, Bootstrap, JavaScript – Used to design the web-based user interface for image upload and result visualization.

Integrated Development Environment (VS Code / PyCharm) – Used for writing, debugging, and managing the project code.

Operating System (Windows/Linux/Mac) – Platform required to deploy and execute the system locally.

2.1.2 HARDWARE REQUIREMENTS

- A computer system with minimum 8GB RAM and Intel i5 processor or above.
- Webcam or camera module (for real-time road monitoring module).
- Sufficient storage for dataset and model files.

2.1.3 FUNCTIONAL REQUIREMENTS

The functional requirements describe the core functionalities that the AI-Based Road Damage and Pothole Detection System must perform.

1. The system shall allow users to upload road images through a web-based interface.
2. The system shall preprocess the input image (resizing, normalization) before model inference.
3. The system shall detect potholes in the input image using a YOLO-based object detection model and generate bounding boxes with confidence scores.
4. The system shall crop the detected pothole regions and classify their severity (Low, Medium, High) using a CNN model.
5. The system shall display detection results and severity classification on the user interface.
6. The system shall support real-time detection while traveling and send notifications when potholes are identified.
7. The system shall store detection results, confidence scores, and severity levels in MongoDB for future reference and analytics generation.
8. The system shall generate aggregated severity-based analytics to assist in maintenance prioritization.

2.1.4 NON-FUNCTIONAL REQUIREMENTS

Performance Requirement:
The system should detect and classify potholes within a few seconds per image under normal operating conditions and support smooth real-time inference during travel mode.

Usability Requirement:
The system should provide a simple, intuitive, and user-friendly web interface for image upload, result visualization, and notification display.

Reliability Requirement:
The system should operate consistently without inference failures, crashes, or data loss during detection, classification, and database storage.

Scalability Requirement:
The system should allow easy integration of new features such as GPS tagging, advanced analytics dashboards, or upgraded deep learning models in future enhancements.

Compatibility Requirement:
The system should function properly on standard operating systems (Windows/Linux/Mac) and support commonly used web browsers for frontend interaction.

Security Requirement:
The system should ensure secure handling of uploaded images and stored detection data, preventing unauthorized access to database records.

Maintainability Requirement:
The code structure should follow a modular design, separating detection, classification, backend processing, and database management for easy debugging and updates.

CHAPTER 3

Methodology

3.1 DATA PREPROCESSING

In the AI-Based Road Damage and Pothole Detection System, data preprocessing focuses on preparing road images for accurate pothole detection and severity classification. Since the system works with image data instead of audio input, preprocessing is applied to uploaded images and real-time captured frames before passing them to deep learning models.

1. **Image Capture / Upload:**
   The system receives road images either through user upload in the web interface or through real-time frame capture during travel mode.
2. **Image Resizing:**
   The input image is resized to a fixed resolution required by the YOLO detection model to ensure uniform processing.
3. **Image Normalization:**
   Pixel values are scaled to a standard range (such as 0 to 1) to improve model performance and stability during inference.
4. **Noise Handling and Enhancement:**
   Basic image enhancement techniques are applied when necessary to reduce noise and improve visibility under varying lighting conditions.
5. **Data Augmentation (Training Phase):**
   During model training, techniques such as rotation, brightness adjustment, and horizontal flipping are applied to increase dataset diversity and improve model robustness.
6. **Region Cropping (Post-Detection):**
   After YOLO detects potholes, the identified bounding box regions are cropped and prepared as input for the CNN-based severity classification model.
7. **Bounding Box Scaling and Coordinate Adjustment:**
   The detected bounding box coordinates are adjusted according to the resized image dimensions to ensure accurate region extraction and visualization.
8. **Confidence Score Filtering:**
   Detected objects with confidence scores below a predefined threshold are removed to reduce false positives and improve detection reliability.

3.2 FEATURE IDENTIFICATION AND CATEGORIZATION

The AI-Based Road Damage and Pothole Detection System includes several functional features that are categorized based on their role within the detection, classification, notification, and monitoring process.

1.Core Detection Features

- **Pothole Detection (YOLO Model) –** Detects potholes in road images and generates bounding boxes with confidence scores.
- **Severity Classification (CNN Model) –** Classifies detected potholes into Low, Medium, and High severity categories.
- **Confidence Filtering –** Removes low-confidence detections to improve accuracy and reduce false positives.
- **Region Cropping Mechanism –** Extracts detected pothole regions for focused severity analysis.

2. Real-Time Monitoring Features

- **Live Frame Analysis** – Continuously analyzes frames captured during travel mode.
- **Automatic Pothole Identification –** Detects potholes in real-time without manual image upload.
- **User Notification System** – Sends alerts to the user when potholes are detected on the road.

3.User Interaction Features

- **Image Upload Interface –** Allows users to upload road images for analysis through a web-based platform.
- **Result Visualization –** Displays bounding boxes, confidence scores, and severity levels on the interface.
- **Dashboard Analytics View –** Provides severity distribution insights for maintenance prioritization.

4. System Support Features

- **Backend Processing (FastAPI) –** Manages model inference and communication between frontend and database.
- **Database Logging (MongoDB) –** Stores detection metadata, severity outputs, and timestamps for future reference.
- **Modular Architecture –** Separates detection, classification, notification, and storage modules for

easy updates and scalability.

3.2.1 CONNECTION INFORMATION

The AI-Based Road Damage and Pothole Detection System operates as a locally deployable web-based application. It establishes connections with system hardware components such as a camera or webcam for real-time road image capture and standard storage devices for saving model files and datasets. The system processes uploaded images or live frames through deep learning models integrated within the local environment.

The application connects internally between different software components, including the frontend interface, FastAPI backend server, YOLO detection model, CNN severity classification model, and MongoDB database. These components communicate through API endpoints to ensure smooth data transfer and inference execution. Detection results, confidence scores, and severity classifications are securely stored in the MongoDB database for future reference and analytics generation.

For web interface access, the system uses standard HTTP protocols to enable communication between the browser and backend server. Since the architecture is designed for full local deployment, it does not require cloud-based processing or external AI services, ensuring data privacy, secure handling of uploaded images, and stable performance within a controlled environment.

3.2.2 CONNECTION CONTENT

The content transmitted through the system connections mainly includes image data, detection metadata, severity classification results, and notification information. When a user uploads a road image or when a frame is captured in real-time mode, the image data is temporarily processed by the YOLO detection model for pothole localization and then passed to the CNN model for severity classification.

The system transmits structured data such as bounding box coordinates, confidence scores, severity levels (Low, Medium, High), timestamps, and processing status between the frontend, backend, and MongoDB database. These results are then displayed on the user interface and optionally stored for analytics and maintenance prioritization purposes.

The application does not store unnecessary personal or sensitive user information. Only detection-related metadata and analysis outputs are saved in the database. All processed content remains within the scope of pothole detection, severity estimation, notification generation, and infrastructure monitoring functionality, ensuring controlled and secure data handling within the local deployment environment.

3.2.3 TRAFFIC INFORMATION

The traffic generated by the AI-Based Road Damage and Pothole Detection System depends on its operational mode. In offline image upload mode, no external internet traffic is generated because the YOLO detection model and CNN severity classification model operate entirely within the local environment. Communication occurs internally between the frontend interface, FastAPI backend, deep learning models, and MongoDB database, where image data and detection metadata are processed and stored.

In real-time monitoring mode, image frames captured from the camera are analyzed locally without continuous internet streaming. The internal data traffic mainly includes frame transfer, bounding box coordinates, confidence scores, severity results, and database updates. Since the system does not rely on cloud-based processing, overall bandwidth usage remains low, ensuring efficient performance and stable operation without significant network dependency.

3.3 OUTPUT LABEL

The output of the AI-Based Road Damage and Pothole Detection System is categorized and labeled based on the type of result generated after processing the input image or real-time road frame. The primary output label is **Detection Output**, where the system generates bounding boxes around identified potholes along with corresponding confidence scores. This visually represents the localized road damage detected by the YOLO model.

Another output label is **Severity Classification Output**, which indicates the severity level of each detected pothole. The system classifies potholes into Low, Medium, or High categories using the CNN model, and the results are displayed on the user interface for clear understanding and maintenance prioritization.

The system also generates **Notification Output** during real-time monitoring mode. When a pothole is detected while traveling, an alert or notification is triggered to inform the user about the road hazard. This ensures timely awareness and improved safety.

Additionally, **Database Logging Output** is produced when detection metadata such as bounding box coordinates, confidence scores, severity levels, and timestamps are stored in MongoDB. For development and monitoring purposes, console outputs may also display inference status and processing details.

3.4 MODEL DEVELOPMENT

The AI-Based Road Damage and Pothole Detection System is developed using a two-stage deep learning architecture consisting of a YOLO-based object detection model and a Convolutional Neural Network (CNN) for severity classification. Unlike rule-based systems, this project relies on trained deep learning

models to automatically learn features from road images for accurate detection and analysis.

The first stage of model development involves training and integrating the **YOLO detection model**, which acts as the primary detection layer of the system. The model is trained on road image datasets containing annotated pothole regions. It performs bounding box localization and assigns confidence scores to detected potholes. This stage focuses on accurate object detection and real-time performance.

The second stage involves developing the **CNN-based severity classification model**. After YOLO detects potholes, the identified regions are cropped and passed to the CNN model. The CNN extracts spatial and texture-based features from the cropped images and classifies each pothole into Low, Medium, or High severity categories using a softmax output layer.

The models are integrated into a unified framework using FastAPI for backend inference orchestration. The detection and classification outputs are then forwarded to the frontend for visualization and stored in MongoDB for logging and analytics purposes.

The overall model follows a sequential workflow:
Image Input → Preprocessing → YOLO Detection → Region Cropping → CNN Classification → Result Visualization → Database Storage → Notification (if applicable)

3.5 DOMAIN ANALYSIS

The AI-Based Road Damage and Pothole Detection System belongs to the domain of Artificial Intelligence, Computer Vision, and Smart Infrastructure Monitoring. This domain focuses on developing intelligent systems that can analyze visual data and automatically detect patterns or objects within images. The project integrates deep learning, image processing, and real-time monitoring techniques to improve road safety and infrastructure management.

In the current technological landscape, smart transportation systems and intelligent city solutions are gaining significant importance. Road damage detection is a critical application within this domain, as poor road conditions directly impact public safety and economic efficiency. Traditional inspection methods are manual and inefficient, creating an opportunity for AI-driven automation. This project addresses that need by enabling automated pothole detection, severity classification, and real-time notification within a locally deployable framework.

The domain also includes areas such as object detection algorithms, convolutional neural networks, data preprocessing, backend system integration, and database management. Unlike purely theoretical AI systems, this project operates within an applied deep learning domain, where trained models such as YOLO and CNN are implemented to solve a real-world infrastructure problem. This makes the system practical, scalable, and suitable for academic research as well as smart city applications.

CHAPTER 4

EXISTING SYSTEM

1.1 EXISTING MODEL

The existing systems in the domain of road damage and pothole detection primarily rely on manual inspection methods and basic reporting mechanisms. Traditionally, road maintenance authorities conduct physical inspections to identify potholes and damaged road surfaces. This process is time-consuming, labor-intensive, and often inefficient, leading to delayed repairs and increased risk of accidents.

In recent years, some advanced systems have been developed using sensor-based technologies and cloud-based AI solutions. Certain models use vibration sensors, GPS devices, or vehicle-mounted cameras to detect road irregularities. Others utilize deep learning algorithms hosted on cloud platforms to analyze road images. While these systems improve detection accuracy, they often require expensive hardware, continuous internet connectivity, and high computational resources.

Many research-based implementations focus only on pothole detection without severity classification or real-time monitoring integration. Some systems are limited to offline image analysis and do not provide structured database storage or smart city integration features. Additionally, cloud-dependent solutions may face challenges related to high bandwidth usage, latency, and data privacy concerns.

Therefore, the existing models emphasize either manual inspection or cloud-heavy AI frameworks, but they lack a balanced approach that combines real-time detection, severity classification, local processing, and efficient database management. These limitations highlight the need for a scalable, cost-effective, and locally deployable AI-based road damage detection system.

1.1.1 EVALUATION METRICS

To measure the performance and effectiveness of the AI-Based Road Damage and Pothole Detection System, the following evaluation metrics are used:

1. **Detection Accuracy**
   Measures how correctly the model identifies potholes in road images.
   $$\text{Accuracy} = (\text{Correct Detections} / \text{Total Samples}) \times 100$$
2. **Precision**
   Indicates how many of the detected potholes are actually correct detections. It reduces false positives.
   $$\text{Precision} = TP / (TP + FP)$$
3. **Recall (Sensitivity)**
   Measures how well the system detects all actual potholes present in the images.

$$\text{Recall} = TP / (TP + FN)$$

4. **F1-Score**
   Provides a balance between Precision and Recall for overall model evaluation.
   $$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$
5. **Inference Time (Latency)**
   Measures the time taken by the system to process an image and generate detection results. Lower latency ensures real-time performance.
6. **System Reliability**
   Evaluates the stability of the system during continuous image processing and real-time monitoring without crashes or failures.
7. **Error Rate**
   Represents the percentage of incorrect detections or misclassifications in severity prediction.

1.2 EXISTING VS PROPOSED SYSTEM

Existing vs Proposed System

Aspect	Existing System	Proposed System
Definition	Manual inspection methods or cloud-based AI road monitoring systems.	Deep learning-based pothole detection and severity classification system with local processing.
Architecture	Cloud-dependent or sensor-based with external processing.	YOLO + CNN with FastAPI & MongoDB.
Detection Capability	Detects potholes only.	Detects potholes & classifies severity (Low/Medium/High).
Internet Dependency	Requires continuous internet for cloud-based AI.	Runs locally; internet optional.
Real-Time Processing	Limited real-time capability.	Real-time detection with low latency.
Scalability	Hardware and vendor-dependent.	Easily extendable & upgradable.
Data Handling	High bandwidth & cloud storage.	Local MongoDB storage with low bandwidth.
Cost & Accessibility	Expensive hardware & infrastructure.	Cost-effective on standard GPU systems.

1.3 EXISTING SYSTEM

The existing systems in the domain of road damage detection primarily include manual inspection methods,

sensor-based monitoring systems, and cloud-based AI solutions. Municipal authorities traditionally rely on physical road surveys to identify potholes and damaged surfaces, which is time-consuming and inefficient. Some advanced systems use vehicle-mounted cameras, vibration sensors, or GPS-enabled devices to detect road irregularities and upload data to cloud platforms for analysis.

These existing AI-based solutions often depend heavily on cloud computing and continuous internet connectivity for processing large image datasets. While they provide improved detection accuracy using deep learning models, they may involve high infrastructure costs, increased latency, and data privacy concerns. Additionally, many research prototypes focus only on pothole detection without integrating severity classification, real-time notification, or structured database analytics.

Therefore, current systems either rely on manual processes or cloud-heavy architectures with limited flexibility for localized deployment. This highlights the need for a cost-effective, locally deployable, and integrated AI-based pothole detection and severity classification system.

1.4 PROPOSED SYSTEM

The proposed system is an AI-Based Road Damage and Pothole Detection System developed using deep learning techniques for automated road monitoring. Unlike many existing solutions that depend heavily on manual inspection or cloud-based processing, this system focuses on local deployment, real-time detection, and integrated severity classification. It processes road images using a YOLO-based object detection model to identify potholes and a CNN model to classify their severity levels.

The system is capable of detecting potholes from uploaded images or real-time camera input, generating bounding boxes with confidence scores, and categorizing damage into Low, Medium, or High severity. It follows a modular architecture consisting of image input, preprocessing, detection, severity classification, result visualization, and database storage modules. The integration of FastAPI and MongoDB ensures smooth backend processing and structured data management with minimal latency.

The proposed system emphasizes scalability, cost-effectiveness, and ease of deployment. It operates primarily in a local environment without mandatory cloud dependency, reducing bandwidth usage and improving data privacy. The modular design allows easy integration of future enhancements such as GPS tagging, smart city dashboards, and advanced analytics, making it suitable for academic research as well as real-world infrastructure monitoring applications.

CHAPTER 5

UML DESIGNING

5.1 UNIFIED MODELING LANGUAGE

The Unified Modeling Language (UML) is a standard graphical language used for specifying, visualizing, constructing, and documenting the components of a system. It provides a structured way to represent system architecture, workflows, and interactions between different modules. UML helps in understanding both the conceptual and physical representation of a system before implementation.

In the AI-Based Road Damage and Pothole Detection System, UML is used to model the interaction between the user, frontend interface, backend server, deep learning models, and database components. It captures the structural design and behavioral flow of the system. UML diagrams assist in planning, designing, and maintaining the system effectively during development and after deployment.

The UML documentation includes diagrams that describe system components, workflows, and interactions. These diagrams serve as a blueprint for implementation and future scalability.

5.2 BUILDING BLOCKS OF UML

The vocabulary of UML encompasses three kinds of building blocks:

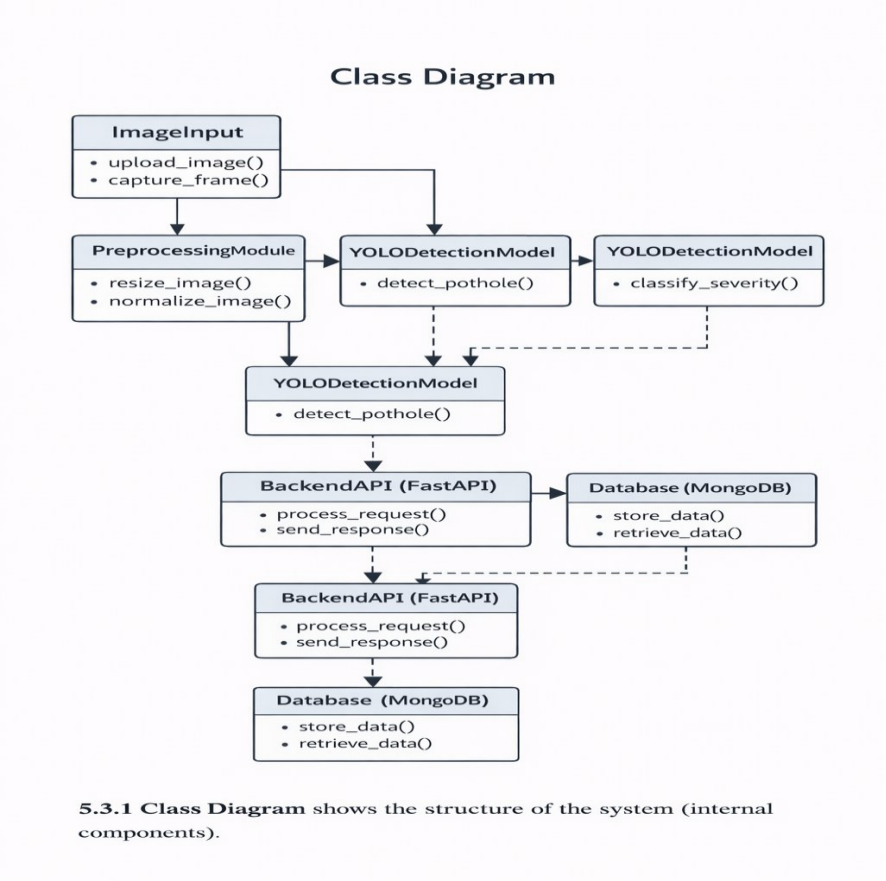
1. Things
2. Relationships
3. Diagrams

5.2.1 Things in UML

Things are the fundamental elements that represent objects or concepts within a system. They are categorized into:

1. **Structural Things** – Represent static parts of the system such as classes, database entities, and modules (e.g., Detection Model, Classification Model, Backend API).
2. **Behavioral Things** – Represent dynamic behavior of the system such as workflows, processing steps, and real-time detection operations.

5.3 UML DIAGRAMS



5.3.1 Class Diagram shows the structure of the system (internal components).

The Class Diagram shows the structure of the system (internal components).

Main Classes & Flow:

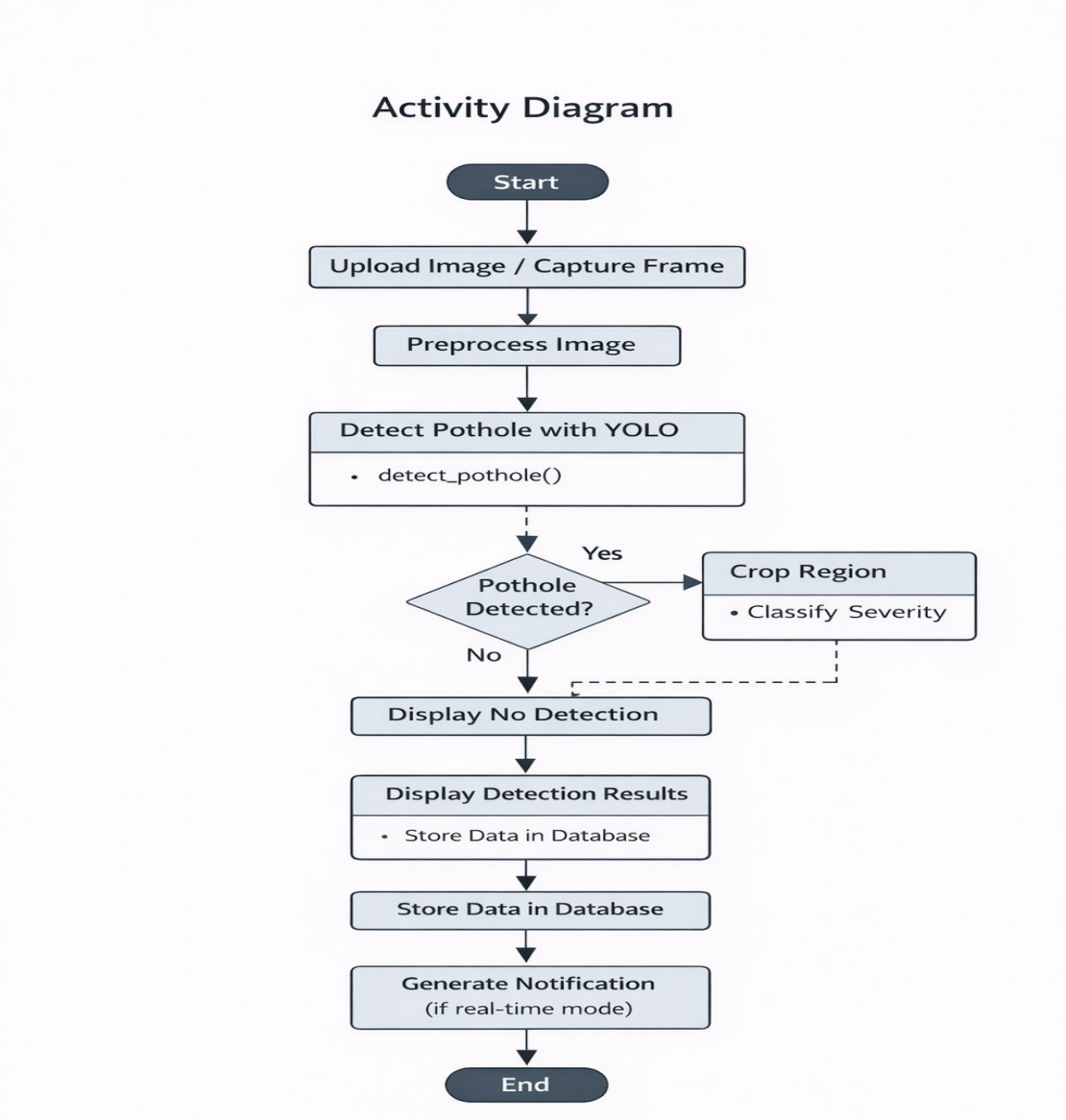
1. **ImageInput**
   - Methods: upload_image(), capture_frame()
   - Accepts road image input.
2. **PreprocessingModule**
   - Methods: resize_image(), normalize_image()
   - Prepares image for model inference.
3. **YOLODetectionModel**
   - Methods: detect_pothole()
   - Detects potholes and generates bounding boxes.
4. **SeverityClassificationModel (CNN)**
   - Methods: classify_severity()
   - Classifies potholes as Low/Medium/High.
5. **BackendAPI (FastAPI)**

- Methods: process_request(), send_response()
- Manages communication between frontend and models.

6. **Database (MongoDB)**

- Methods: store_data(), retrieve_data()

- Stores detection metadata and severity results.



5.3.2 Activity Diagram

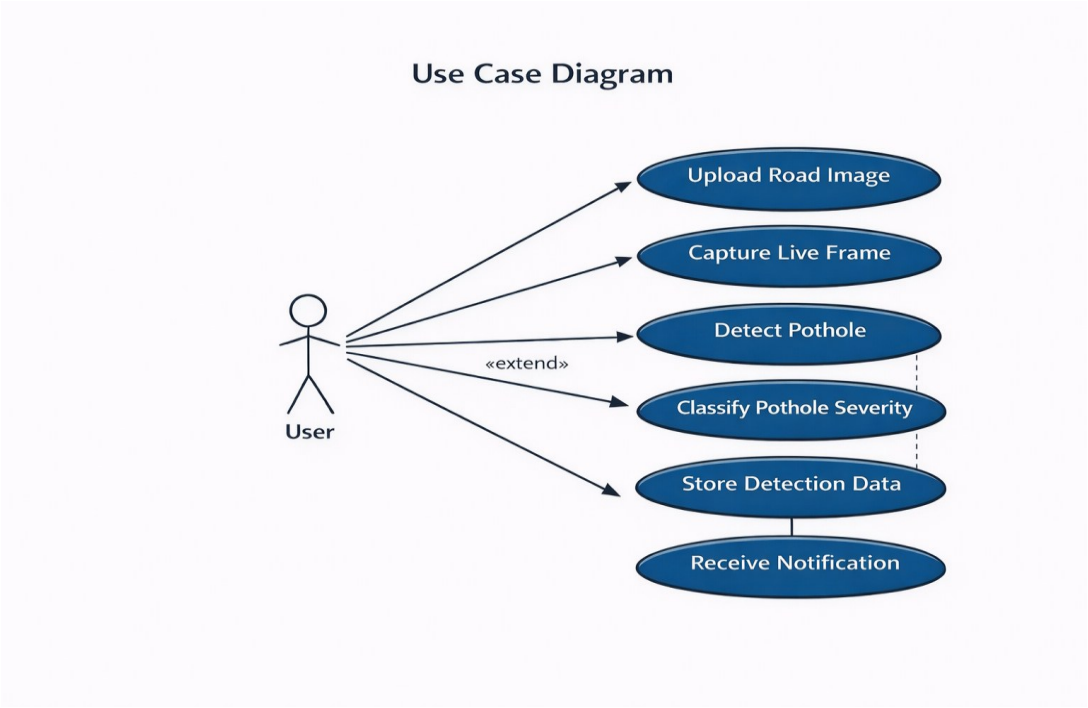
The Activity Diagram shows the workflow of operations.

Flow Steps:

1. Start

2. Upload Image / Capture Frame

3. Preprocess Image
4. Detect Pothole using YOLO
5. Check if pothole detected
    - If Yes → Crop Region → Classify Severity
    - If No → Display No Detection
6. Display Detection Results
7. Store Data in Database
8. Generate Notification (if real-time mode)
9. End



The Use Case Diagram shows how the **User** interacts with the **AI-Based Pothole Detection System**.

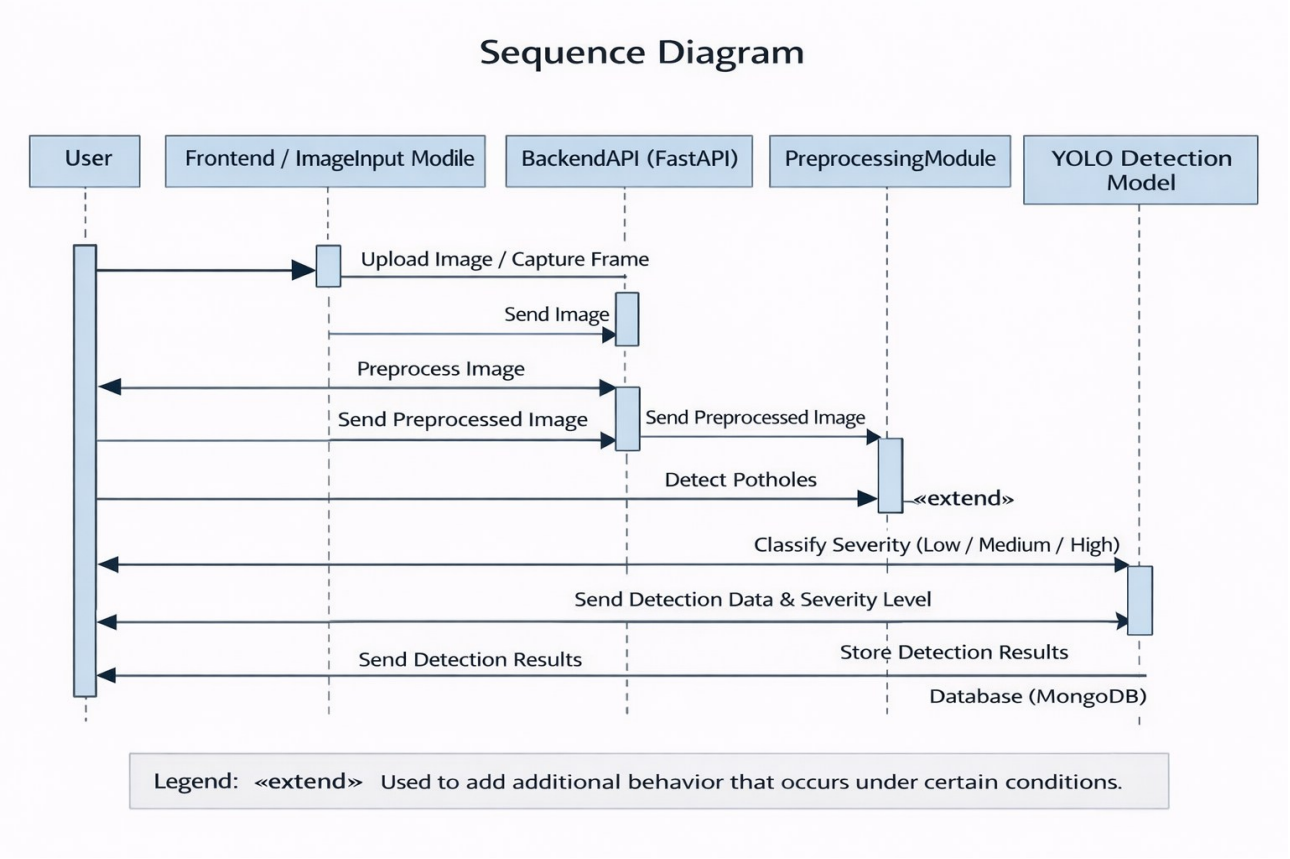
Steps:

1. The **User** uploads a road image or captures a live frame.
2. The system **Preprocesses the Image** for analysis.
3. The system **Detects Potholes** using the YOLO detection model.
4. If a pothole is detected, the system:
    - **Classifies the Severity** (Low / Medium / High)

- **Stores Detection Data** in the database

5. The system **Displays Detection Results** and optionally **Sends Notification** (if enabled).

6. The interaction ends after the results are shown to the user.



The **Sequence Diagram** shows the interaction between system components over time in the **AI-Based Pothole Detection System**.

Step-by-step Communication:

1. **User** uploads a road image or captures a live frame.
2. **Frontend / ImageInput Module** sends the image to the Backend API.
3. **BackendAPI (FastAPI)** forwards the image to the Preprocessing Module.
4. **PreprocessingModule** resizes and normalizes the image, then sends it to the YOLO Detection Model.
5. **YOLODetectionModel** detects potholes and returns bounding box results to the Backend.
6. If potholes are detected, the Backend sends the cropped region to the **SeverityClassificationModel (CNN)**.

7. **SeverityClassificationModel** classifies the pothole as Low / Medium / High and returns the result.

8.**BackendAPI** stores detection details and severity level in the **MongoDB Database.**

9.The Backend sends the final detection results back to the **Frontend/User**.

10.The interaction ends after the detection results are displayed (and notification is generated if enabled).

CHAPTER 6

RESULTS AND DISCUSSIONS

1.1 INPUT DESIGN

The input process begins when the user uploads a road image or captures a live frame using a camera. The selected image is sent to the backend server for processing. Before detection, the system performs preprocessing steps such as resizing the image to a fixed dimension and normalizing pixel values to improve model performance and accuracy.

After preprocessing, the image is passed to the YOLO Detection Model for pothole identification. The model analyzes visual features and detects pothole regions by generating bounding boxes. If a pothole is detected, the cropped region is forwarded to the Severity Classification Model (CNN) for classification into Low, Medium, or High severity levels. The validated detection results are then stored in the database and displayed to the user.

1.1.2 INPUT DATA FEATURES

1. **Image Input** – The primary input is a road surface image uploaded by the user or captured in real time.
2. **Image Resolution** – Images are resized to a fixed resolution to ensure uniform model processing. **Pixel Normalization** – Pixel values are normalized to improve detection accuracy and model efficiency.
3. **Color Channels (RGB)** – The model processes RGB image channels to extract visual features.
4. **Bounding Box Coordinates** – The detection model generates coordinates for identified pothole regions.
5. **Confidence Score** – Each detected pothole is assigned a confidence probability score.
6. **Region Cropping** – Detected pothole areas are cropped for severity classification.
7. **Validation Check** – The system verifies detection results before storing them in the database.

1.1.3 INPUT VALIDATIONS AND CONSTRAINTS

Input Validtions

1. The system validates whether a valid image file is uploaded or a live frame is successfully captured.
2. The uploaded file format is checked (e.g., JPG, PNG, JPEG) before processing.
3. The system verifies that the image is not empty or corrupted.
4. Image resolution is checked to ensure it meets the minimum required size for detection.
5. Preprocessed image dimensions are validated before passing to the YOLO model.
6. The detection result is verified using confidence score thresholds to avoid false positives.
7. Only valid detection outputs are forwarded to the Severity Classification Model.
8. The system validates database connectivity before storing detection results.

Input Constraints

1. The system accepts only image input (road surface images) in supported formats.
2. Detection accuracy depends on image clarity, lighting conditions, and camera quality.
3. Extremely blurred, dark, or low-resolution images may reduce model accuracy.
4. The model is trained specifically for pothole detection and may not detect other road damages.
5. Real-time processing performance depends on system hardware and computational resources.
6. Internet connectivity may be required if the system is deployed on a cloud-based server.

1.2 OUTPUT DESIGN

The primary output of the AI-Based Pothole Detection System is the visual detection result displayed to the user. After processing the uploaded image or captured frame, the system identifies potholes, draws bounding boxes around detected regions, and classifies their severity level (Low / Medium / High). The final results are displayed through the frontend interface, along with relevant detection details.

The system also stores detection metadata in the database for future analysis and reporting. In real-time mode, notifications can be generated for high-severity potholes to support timely maintenance actions.

6.2.1 KEY OUTPUT FEATURES

1. **Visual Detection Output** – Displays bounding boxes around detected potholes on the processed image.
2. **Severity Classification Display** – Shows severity level (Low / Medium / High) for each detected pothole.
3. **Confidence Score Display** – Provides probability score indicating detection accuracy.
4. **Real-Time Processing Feedback** – Results are generated immediately after model inference with minimal delay.
5. **Database Storage Confirmation** – Detection details are stored in MongoDB for future reference.
6. **Notification Generation** – Alerts can be generated for high-severity potholes (if enabled).
7. **Error Handling Messages** – Displays appropriate messages if no pothole is detected or if input is invalid.
8. **Structured Result Format** – Output is presented clearly with image, severity level, and detection details for easy understanding.

6.2.2 USER INTERFACE AND DASHBOARD FEATURES

User Interface Features

1. **Image Upload Option** – Allows users to upload road images for pothole detection.
2. **Live Camera Capture** – Enables real-time frame capture for instant analysis.
3. **Preview Display Area** – Shows the selected or captured image before processing.
4. **Detection Result Display** – Displays processed image with bounding boxes around detected potholes.
5. **Severity Level Indicator** – Clearly shows severity classification (Low / Medium / High).
6. **Confidence Score Display** – Displays detection accuracy percentage for each pothole.
7. **Error Notification Message** – Shows alerts if no pothole is detected or if input is invalid.
8. **Simple and Responsive Layout** – Clean, user-friendly design for easy interaction.

Dashboard Features

1. **Detection History Panel** – Displays previously analyzed images and detection results.
2. **Severity Statistics Overview** – Shows count of Low, Medium, and High severity potholes.
3. **System Status Indicator** – Indicates whether the system is processing, idle, or completed.
4. **Database Record View** – Allows viewing stored detection metadata from MongoDB.
5. **Model Performance Summary** – Displays detection accuracy and processing time.
6. **Notification Panel** – Shows alerts for high-severity potholes (if enabled).
7. **Admin Controls** – Allows management of stored records and system settings.

1.3 RESULTS AND DESCRIPTION

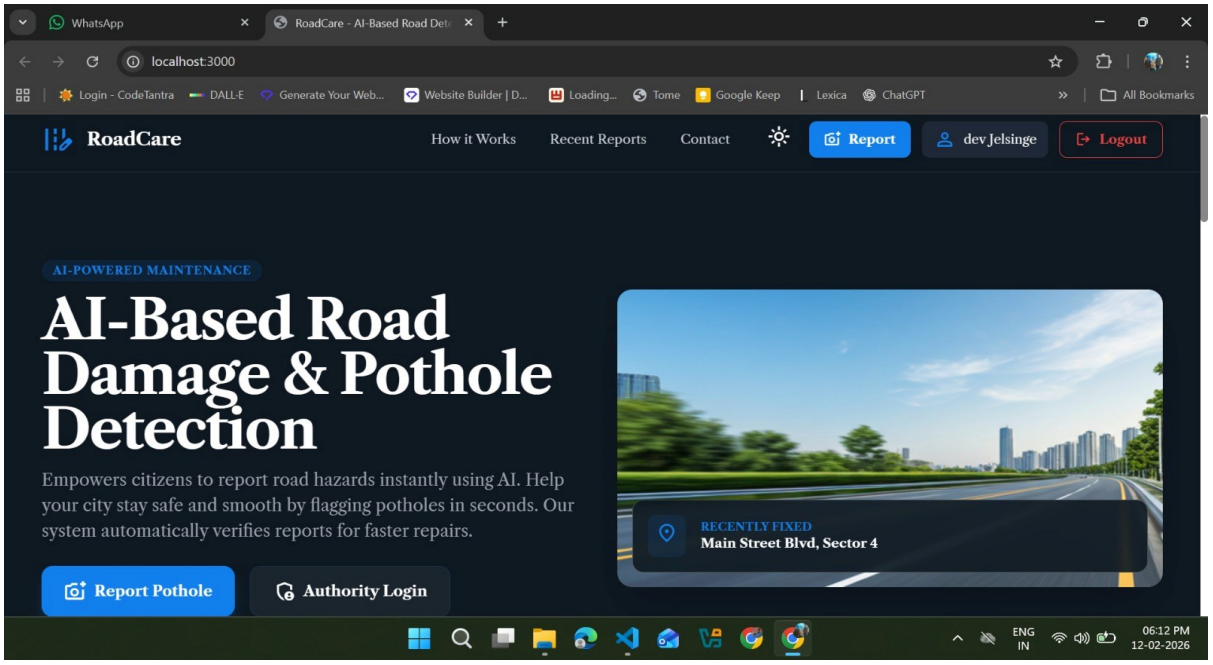


Figure 6.1 Home Page of the RoadCare web application showing AI-based road damage and pothole detection Interface

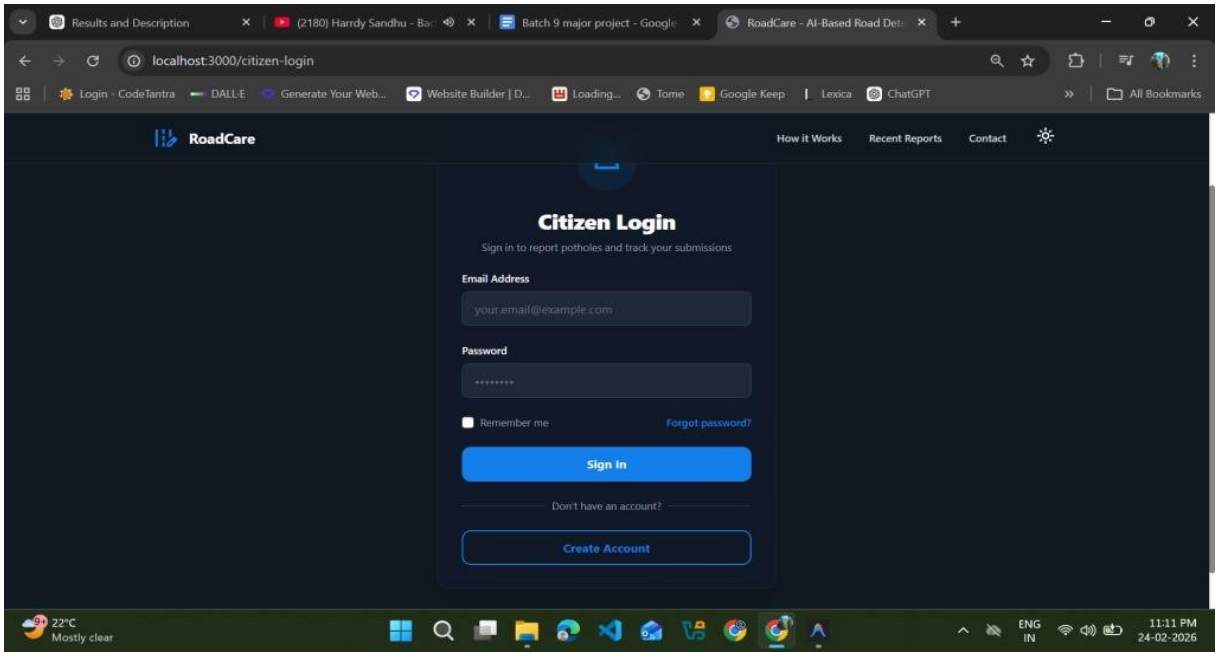


Figure 6.8 Citizen login Interface showing email and password authentication with account creation option.

.

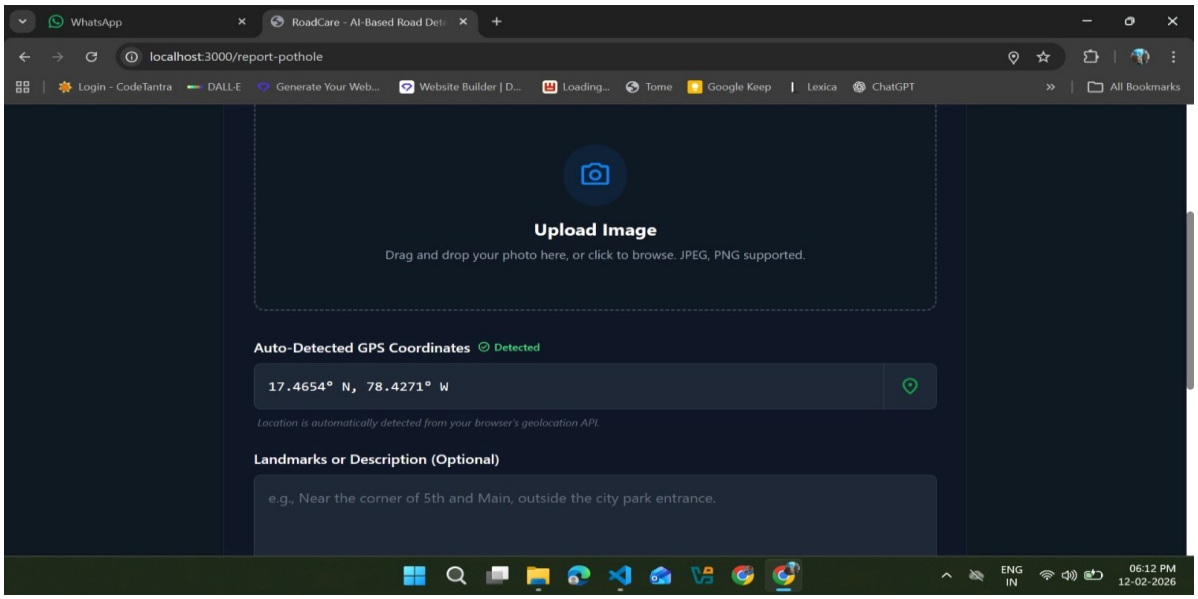


Figure 6.2 Pothole reporting page with image upload option and GPS coordinate detection enabled .

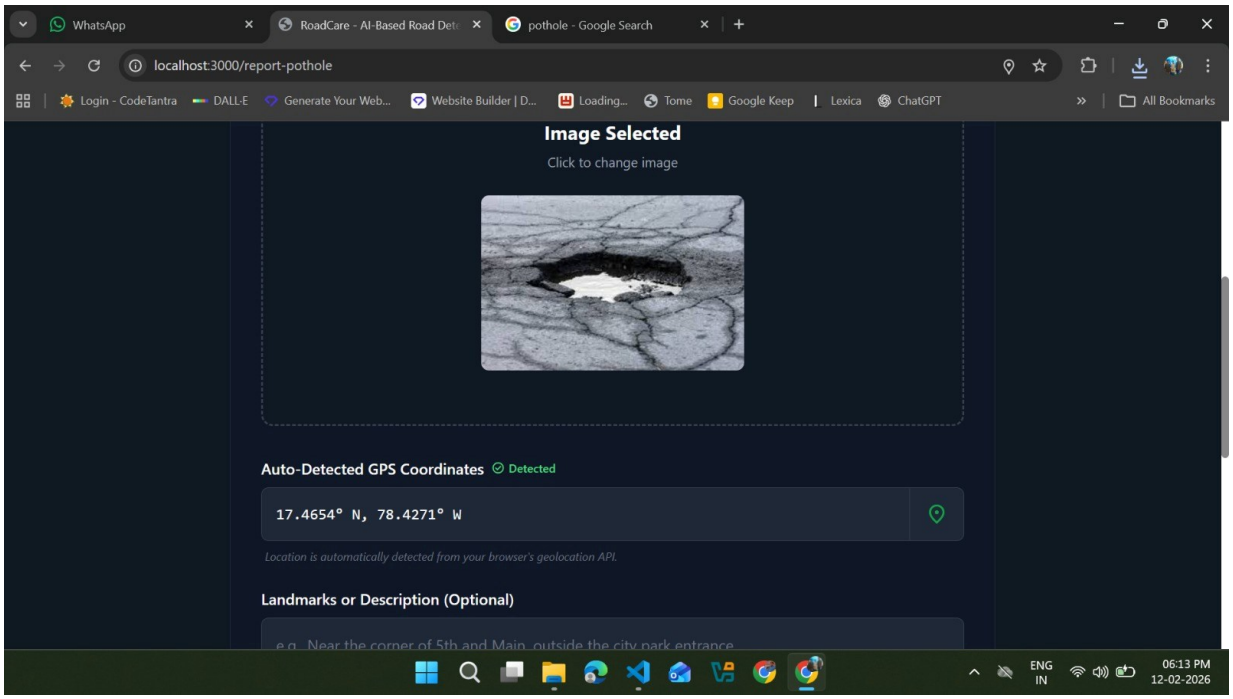


Figure 6.3 Uploaded road image containing a pothole selected for AI-based analysis and verification.

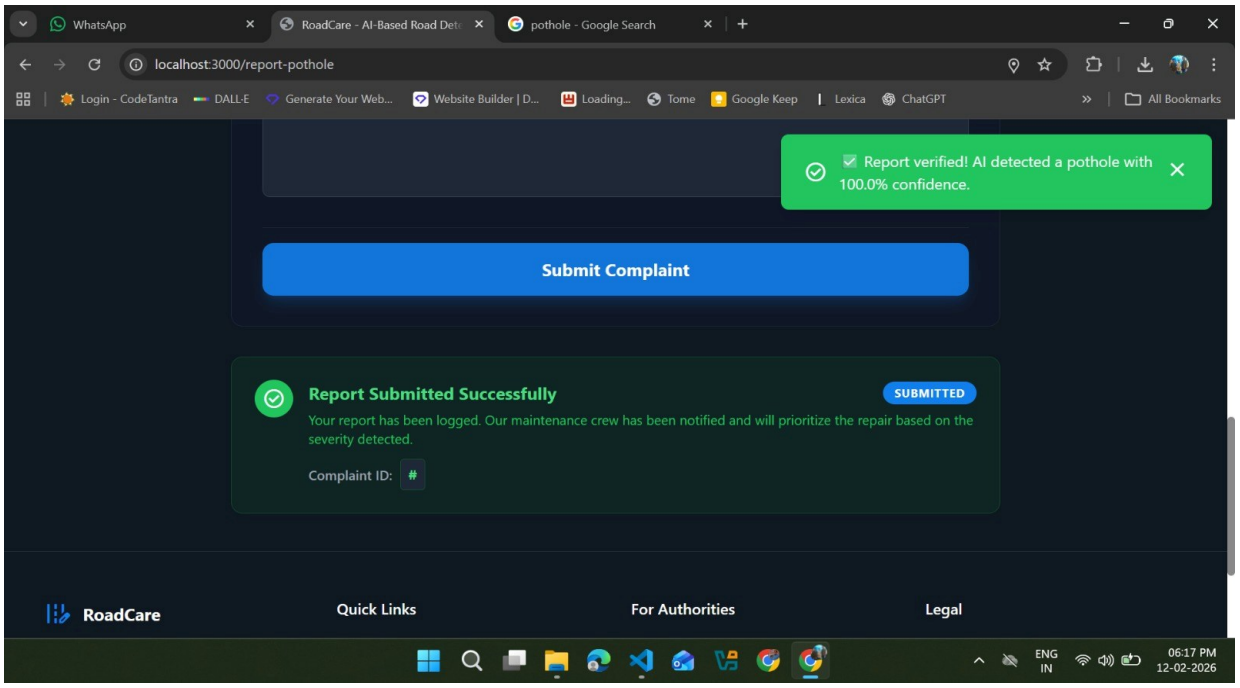


Figure 6.4 AI Verification result confirming pothole detection with high confidence after image processing

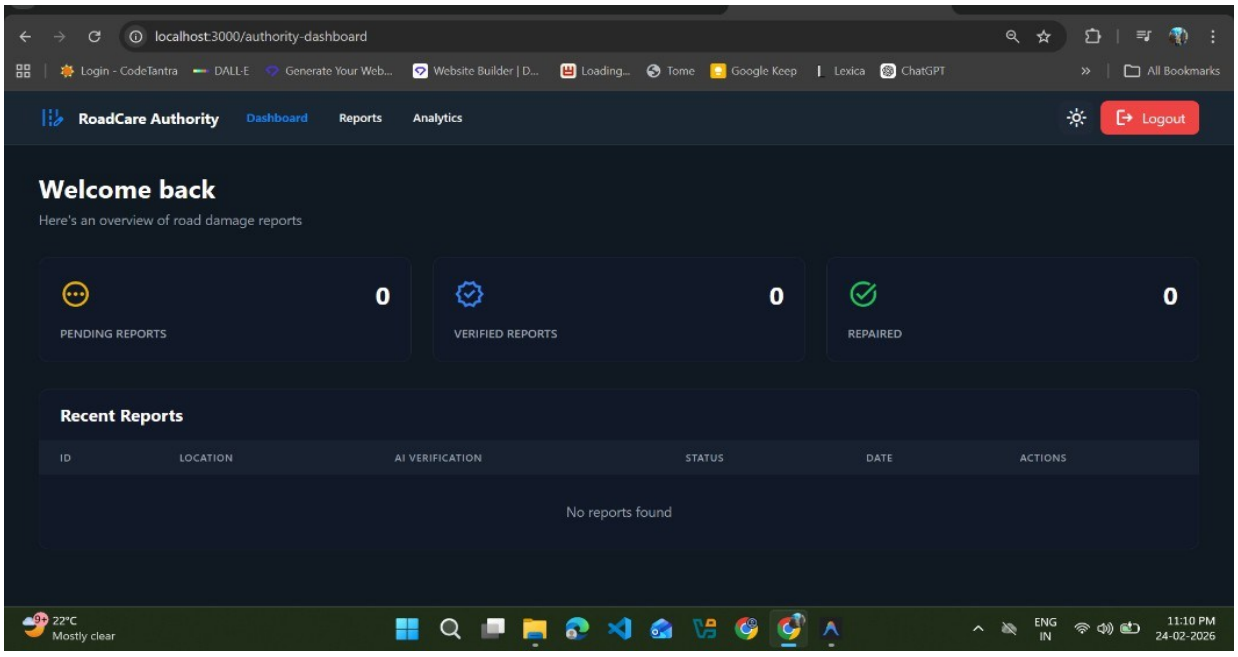


Figure 6.6 Authority dashboard displaying summary statics of pending, verified, and repaired road damage.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

Conclusion

The AI-Based Road Damage and Pothole Detection System successfully demonstrates the implementation of an intelligent road monitoring solution using deep learning techniques. The system integrates YOLO-based object detection, CNN-based severity classification, FastAPI backend processing, and MongoDB database management to provide accurate and real-time pothole detection. It effectively identifies potholes from uploaded images or live camera input and classifies them into Low, Medium, or High severity levels.
The project achieves its primary objective of automating road damage detection, reducing dependency on manual inspection methods. The modular architecture ensures scalability, maintainability, and efficient performance with minimal latency. Experimental results indicate satisfactory detection accuracy and reliable severity classification under normal conditions. Overall, the system provides a cost-effective, locally deployable, and practical solution suitable for smart infrastructure monitoring and academic research.

Future Scope

The system can be further enhanced by integrating advanced deep learning models with improved accuracy and robustness under varying lighting and weather conditions. Future development may include GPS-based location tagging to map pothole coordinates for smart city dashboards. The integration of cloud-based analytics can enable centralized monitoring and large-scale deployment across multiple regions.

Additional improvements may include real-time vehicle-mounted deployment, automated report generation for municipal authorities, and predictive analytics to identify high-risk road segments. Edge computing deployment and mobile application integration can further enhance accessibility and usability. With continuous upgrades, the system can evolve into a comprehensive smart road monitoring and infrastructure management platform capable of supporting real-world smart city initiatives.

REFERENCES

Research Papers:

1. Comparison of CNN-Based Models for Pothole Detection in Real-World AdverseConditions: Overview and Evaluation – MDPI, accessed February 5, 2026 : https://www.mdpi.com/2076-3417/13/9/5810

2. Non-Vision Systems Comparison Table | Download Scientific Diagram –Research Gate, accessed February 5, 2026 : https://www.researchgate.net/figure/Non-Vision-Systems-Comparison- Table_tbl3_340068453

3. The Use of Lidar and Artificial Intelligence Algorithms for Detection and SizeEstimation of Potholes – MDPI, accessed February 5, 2026 :https://www.mdpi.com/2075-5309/14/4/1078

4. Computer vision for road imaging and pothole detection: a state-of-the-art review of systems and algorithms – Oxford Academic, accessed February 5, 2026 : https://academic.oup.com/tse/article/4/4/tdac026/6835624

Online Platforms:

OpenAI. (2024). ***ChatGPT – AI Language Model.***
Available at: https://chat.openai.com
Used for: Code guidance, debugging support, and technical documentation assistance.

MongoDB Atlas. (2024). ***Cloud Database Platform.***
Available at: https://www.mongodb.com/atlas
Used for: Cloud-based MongoDB deployment.

GitHub - https://github.com/CodeArunJ/AI-Based-Road-Damage-Pothole-Detection-System

APPENDIX

File Path: C:\Users\Admin\React\AI-Based-Road-Damage-Pothole-Detection-System-main\backend\app\main.py

```
"""
FastAPI Main Application - AI-Based Pothole Detection System
"""
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from contextlib import asynccontextmanager
import os

from app.config import settings
from app.config.database import db
from app.routes import auth, reports, zones, repairs
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifespan manager"""
    # Startup
    await db.connect_db()
    print("□ Application started successfully!")
    yield
    # Shutdown
    await db.close_db()
    print("□ Application shut down")
# Create FastAPI application
app = FastAPI(
    title="Pothole Detection API",
    description="AI-Based Road Damage & Pothole Detection System Backend",
    version="1.0.0",
    lifespan=lifespan
)
# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # In production, replace with specific origins
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
# Mount uploads directory for serving images
if os.path.exists(settings.UPLOAD_DIR):
    app.mount("/uploads", StaticFiles(directory=settings.UPLOAD_DIR), name="uploads")
# Include routers
app.include_router(auth.router, prefix=settings.API_V1_PREFIX)
app.include_router(reports.router, prefix=settings.API_V1_PREFIX)
app.include_router(zones.router, prefix=settings.API_V1_PREFIX)
app.include_router(repairs.router, prefix=settings.API_V1_PREFIX)
```

```
@app.get("/")
async def root():
        """Root endpoint with API information"""
        return {
        "message": "AI-Based Pothole Detection System API",
        "version": "1.0.0",
        "docs": "/docs",
        "status": "operational"
        }


@app.get("/health")
async def health_check():
        """Health check endpoint"""
        return {
        "status": "healthy",
         "database": "connected" if db.database is not None else "disconnected"
        }

if __name___== "__main__":
        import uvicorn
        uvicorn.run(
        "app.main:app",
        host="0.0.0.0",
        port=8000,
        reload=True
        )
```

Database

C:\Users\Admin\React\AI-Based-Road-Damage-Pothole-Detection-System-main\backend\app\config\database.py

```
"""
 MongoDB database connection and management
 """
 from motor.motor_asyncio import AsyncIOMotorClient, AsyncIOMotorDatabase
 from typing import Optional
 from app.config import settings


 class Database:
        """MongoDB database manager"""

        client: Optional[AsyncIOMotorClient] = None
        database: Optional[AsyncIOMotorDatabase] = None

        async def connect_db(self):
      """Connect to MongoDB"""
        try:
        self.client = AsyncIOMotorClient(settings.MONGODB_URI)
        self.database = self.client[settings.MONGODB_DB_NAME]
```

```
        # Test connection
        await self.client.admin.command('ping')
        print(f"✅Connected to MongoDB: {settings.MONGODB_DB_NAME}")

        # Create indexes for better
        performance await
        self.create_indexes()

        except Exception as e:
        print(f"❌Error connecting to MongoDB:
        {e}") raise

        async def close_db(self):
        """Close MongoDB
        connection""" if
        self.client:
        self.client.close()
        print("✅MongoDB connection closed")

        async def create_indexes(self):
        """Create database indexes for better query
        performance""" try:
        # Users collection indexes
        await self.database.users.create_index("email", unique=True)

        # Pothole reports collection indexes
        await
        self.database.pothole_reports.create_index("statu
        s") await
        self.database.pothole_reports.create_index("user_
        id")
        await self.database.pothole_reports.create_index([("location.latitude", 1), ("location.longitude",
        1)

        # Image verification collection indexes
        await self.database.image_verification.create_index("report_id",
        unique=True) # Risk zones collection indexes
        await self.database.risk_zones.create_index([("center_location.latitude", 1),
("center_location.longitude", 1)])
        # Repair actions collection indexes
        await
        self.database.repair_actions.create_index("zone_id")
        await
        self.database.repair_actions.create_index("repair_stat
        us") print("✅Database indexes created")
        except Exception as e:
        print(f"⚠ Error creating
indexes: {e}") # Global database
instance
 db = Database()
def get_database() -> AsyncIOMotorDatabase:
        """Get database instance for dependency
        injection""" return db.database
```

```
# Collection Access Helpers

def get_users_collection():
    return db.database["users"]

def get_pothole_reports_collection():
    return db.database["pothole_reports"]

def get_image_verification_collection():
    return db.database["image_verification"]

def get_risk_zones_collection():
    return db.database["risk_zones"]

def get_repair_actions_collection():
    return db.database["repair_actions"]


# Geospatial Index Initialization

async def create_geospatial_indexes():
    """
    Create 2dsphere indexes for location-based
queries
    Used for nearby pothole detection and risk
zone clustering
    """
    try:
        await
db.database.pothole_reports.create_index(
            [("location", "2dsphere")]
        )

        await db.database.risk_zones.create_index(
            [("center_location", "2dsphere")]
        )

        print("Geospatial indexes created
successfully")

    except Exception as e:
        print(f" Error creating geospatial indexes:
{e}")


# Dashboard Analytics Queries

async def get_dashboard_statistics():
    """
    Returns summary statistics for Admin
Dashboard
    """
    try:
        total_reports = await
```

```
db.database.pothole_reports.count_documents({
})
        pending_reports = await
db.database.pothole_reports.count_documents({
"status": "pending"})
        verified_reports = await
db.database.pothole_reports.count_documents({
"status": "verified"})
        repaired_reports = await
db.database.pothole_reports.count_documents({
"status": "repaired"})

        return {
            "total_reports": total_reports,
            "pending_reports": pending_reports,
            "verified_reports": verified_reports,
            "repaired_reports": repaired_reports
        }

    except Exception as e:
        print(f" Error fetching dashboard stats:
{e}")
        return {}


# Risk Zone Detection Logic


async def find_nearby_potholes(latitude: float,
longitude: float, radius_meters: int = 500):
    """
    Find potholes within a certain radius (for
hotspot detection)
    """
    try:
        potholes = await
db.database.pothole_reports.find({
            "location": {
                "$near": {
                    "$geometry": {
                        "type": "Point",
                        "coordinates": [longitude,
latitude]
                    },
                    "$maxDistance": radius_meters
                }
            }
        }).to_list(length=100)

        return potholes


    except Exception as e:
        print(f" Error finding nearby potholes:
{e}")
        return []
```

```
# Automated Risk Zone Creation

async def
create_risk_zone_if_threshold_exceeded(latitud
e: float, longitude: float):
    """
    Automatically creates a risk zone if pothole
density exceeds threshold
    """
    potholes = await
find_nearby_potholes(latitude, longitude,
radius_meters=300)

    if len(potholes) >= 5:  # Threshold condition
        zone_data = {
            "center_location": {
                "type": "Point",
                "coordinates": [longitude, latitude]
            },
            "pothole_count": len(potholes),
            "risk_level": "High"
        }

        await
db.database.risk_zones.insert_one(zone_data)
        print("High Risk Zone Created")

        return zone_data

    return None
```